

LEARN PYTHON PROGRAMMING – BEGINNER TO MASTER

Section: 6. Loops - Control Flow

Lecture: 63. More about for Loop

Section 1: Lecture Summary

This lecture expands on Python's for loop by introducing and explaining four important control flow statements that can be used within a for loop: **break**, **continue**, **pass**, and the **else** clause. You already know break and continue from while loops; here, you learn how they work within for loops. The instructor emphasizes their use cases, behaviors, and how these control statements affect loop execution and termination. A new concept, pass, is introduced as a placeholder when no code is to be executed inside the loop body. Lastly, the else clause of a for loop is explained: it runs only if the for loop completes all iterations normally without interruption by break. The lecture is theoretical, providing example pseudocode to illustrate how these statements alter the flow within for loops so you can apply them in your programs when needed.

Section 2: Key Concepts and Explanations

- For Loop Basics

- Syntax: `for i in range(0, n):`
- Loops over `i` from 0 up to `n-1`.
- Runs the loop body `n` times if no interruption occurs.

- Break Statement

- Purpose: Immediately exit the for loop, stopping any further iterations.
- Used within a conditional statement inside the loop.

- If a break is encountered, the loop terminates prematurely (early exit).
- Example logic flow:
 - For each iteration, execute statements.
 - If a condition is met (true), execute `break`.
 - The loop stops; subsequent iterations are skipped.
- Terminology: Loop terminates unsuccessfully if broken early.
- Works the same as in a while loop.

- Continue Statement

- Purpose: Skip the rest of the current loop iteration and immediately start the next iteration.
- Used within a conditional inside the loop.
- When `continue` executes:
 - Remaining statements in current iteration are skipped.
 - Loop control goes back to the top for the next value.
- Useful for bypassing certain actions conditionally.

- Pass Statement

- Purpose: A no-operation (NOP) placeholder.
- Required when a loop (or any block) cannot be left empty syntactically.
- Example Use Case: Writing a loop body that intentionally does nothing (e.g., to create a delay).
- Can be used with for, while, if, else blocks.
- Does not affect loop execution; the loop still runs all iterations doing nothing.

- Else Clause with For Loop

- Syntax:

```
for i in range(0, n):  
    ...  
else:  
    ...
```

- The else block executes only if the loop completes all iterations without hitting a `break`.
- If the loop is terminated early due to `break`, the else block is skipped.
- This behavior is analogous to the else clause in while loops.
- Useful to distinguish between normal completion and early termination of loops.

Section 3: Example Code and Use Cases

- Break Statement Example

```
for i in range(0, n):  
    print(f"Iteration {i}")  
    if i == 3: # Condition to break early  
        break  
    print("Continuing loop")  
print("Loop terminated")
```

Explanation: The loop prints iteration number and stops when `i` equals 3, exiting without completing all iterations.

- Continue Statement Example

```
for i in range(0, n):  
    print(f"Start iteration {i}")  
    if i % 2 == 0: # Condition to skip rest of loop on even numbers  
        continue
```

```
print(f"Odd iteration {i} - executed")
print("Loop finished")
```

Explanation: For even `i`, the loop skips printing "Odd iteration" by using `continue`, jumping to next iteration immediately.

- Pass Statement Example

```
for i in range(0, n):
    pass
print("Loop repeated n times doing nothing")
```

Explanation: The loop runs `n` times but does nothing inside. `pass` is used so syntax remains valid.

- For-Else Statement Example

```
for i in range(0, n):
    print(f"Checking {i}")
    if i == 5:
        print("Breaking loop!")
        break
else:
    print("Loop completed all iterations without break")
print("After loop")
```

Explanation: If the loop reaches `i == 5`, it breaks and skips the else block. Otherwise, if loop runs fully, the else block executes.

Section 4: Key Takeaways

- The **break** statement stops a for loop before it finishes all iterations.

- The **continue** statement skips the remainder of the current iteration and moves to the next.
- The **pass** statement acts as a placeholder inside loop bodies or other blocks when no code is needed.
- The **else** clause in a for loop runs only if the loop completes without encountering a break.
- These statements help control the execution flow inside for loops, enabling conditional iteration control.
- Understanding these flow control tools is essential for writing flexible and readable loops in Python.

Lecture in Slides

Please Note: This document presents a curated selection of slides from the lecture; others have been excluded for conciseness.



More about for Loop

More about for Loop

- break

More about for Loop

- break
- continue

More about for Loop

- break
- continue
- pass

More about for Loop

- break
- continue
- pass
- else suite

- break

break Statement

break Statement

```
for i in range(0,n):
```

```
    Statement 1
```

```
    Statement 2
```

```
Statement 3
```

break Statement

```
for i in range(0,n):
```

```
    Statement 1
```

```
    Statement 2
```

```
    Statement 3
```

break Statement

```
for i in range(0,n):
```

```
    Statement 1
```

```
    if <condition> :
```

```
        Statement 2
```

```
    Statement 3
```

break Statement

```
for i in range(0,n):
```

```
    Statement 1
```

```
    if <condition> :
```

```
        break
```

```
    Statement 2
```

```
Statement 3
```

break Statement

```
➡➡➡ for i in range(0,n):  
    Statement 1  
    if <condition> :  
        break  
    Statement 2  
Statement 3
```

break Statement

```
for i in range(0,n):  
➡➡➡ Statement 1  
    if <condition> :  
        break  
    Statement 2  
Statement 3
```

break Statement

```
for i in range(0,n):  
    Statement 1  
    ➡ if <condition> :  
        break  
    Statement 2  
Statement 3
```

break Statement

```
for i in range(0,n):  
    Statement 1  
    ➡ if <condition> :  
        break  
    Statement 2  
Statement 3
```

FALSE

break Statement

```
for i in range(0,n):
```

```
    Statement 1
```

```
    if <condition> :
```

```
        break
```

```
    Statement 2
```

```
Statement 3
```

break Statement

```
for i in range(0,n):
```

```
    Statement 1
```

```
    if <condition> :
```

```
        break
```

```
    Statement 2
```

```
Statement 3
```

break Statement

```
for i in range(0,n):
```

```
    Statement 1
```

```
    if <condition> :
```

```
        ➡ break
```

```
    Statement 2
```

```
Statement 3
```

break Statement

```
for i in range(0,n):
```

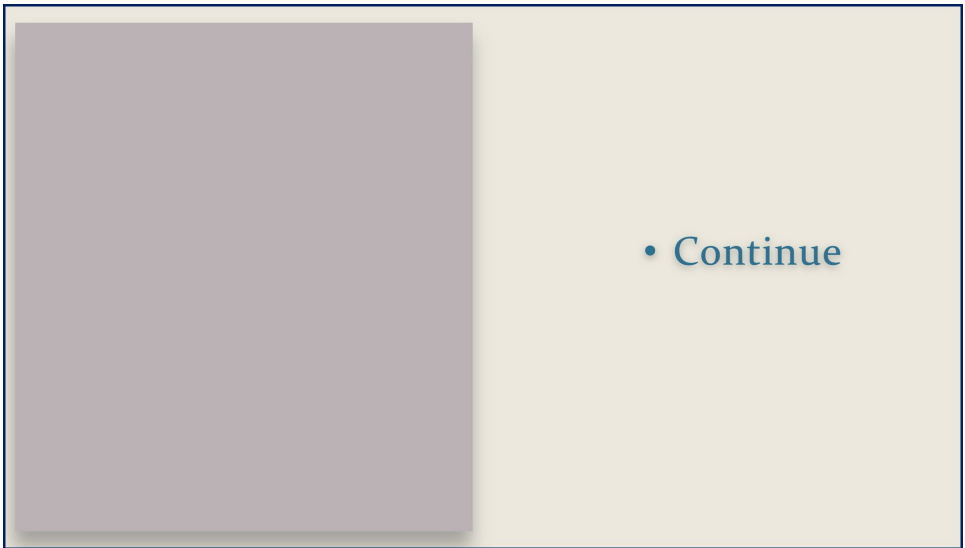
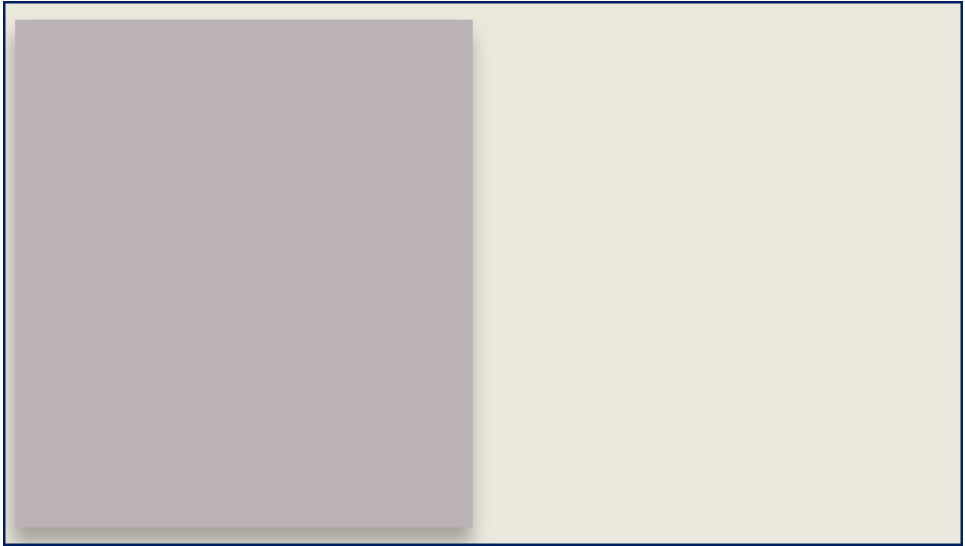
```
    Statement 1
```

```
    if <condition> :
```

```
        break
```

```
    Statement 2
```

```
➡ Statement 3
```

Continue Statement

Continue Statement

```
for i in range(0,n):
```

```
    Statement 1
```

```
    Statement 2
```

```
    Statement 3
```

Continue Statement

```
for i in range(0,n):
```

```
    Statement 1
```

```
    Statement 2
```

```
    Statement 3
```

Continue Statement

```
for i in range(0,n):
```

```
    Statement 1
```

```
    if <condition> :
```

```
        Statement 2
```

```
    Statement 3
```

Continue Statement

```
for i in range(0,n):
```

```
    Statement 1
```

```
    if <condition> :
```

```
        continue
```

```
    Statement 2
```

```
    Statement 3
```

Continue Statement

```
for i in range(0,n):  
    Statement 1  
    if <condition> :  
        continue  
    Statement 2  
Statement 3
```

Continue Statement

```
for i in range(0,n):  
    Statement 1  
    if <condition> :  
        continue  
    Statement 2  
Statement 3
```

```
for i in range(0,n):
    Statement 1
    if <condition>:
        continue
    Statement 2
Statement 3
```

```
for i in range(0,n):
    Statement 1
    if <condition>:
        continue
    Statement 2
Statement 3
```

Continue Statement

```
for i in range(0,n):  
    Statement 1  
    ➡ if <condition> :  
        continue  
    Statement 2  
    Statement 3
```

Continue Statement

```
for i in range(0,n):  
    Statement 1  
    ➡ if <condition> :  
        continue  
    Statement 2  
    Statement 3
```

Continue Statement

```
for i in range(0,n):  
    Statement 1  
    if <condition> :  
        continue  
    ➡ Statement 2  
    Statement 3
```

Continue Statement

```
for i in range(0,n):  
    Statement 1  
    ➡ if <condition> :  
        continue  
    Statement 2  
    Statement 3
```

Continue Statement

```
for i in range(0,n):  
    Statement 1  
    if <condition> :  
        ➡ continue  
    Statement 2  
    Statement 3
```


Continue Statement

```
for i in range(0,n):  
    Statement 1  
    if <condition> :  
        continue  
    Statement 2  
    ➡ Statement 3
```

- pass

Pass Statement

Pass Statement

```
for i in range(0,n):
```

Pass Statement

```
for i in range(0,n):  
    pass
```





• else suite



else Suite





